

# A DevSecOps Security Monitoring Architecture for LogiFlow: From Vulnerable Baseline to Continuous Threat Visibility

Andersson David Sanchez Mendez, Cristian Santiago Pedraza Rodriguez, Jeisson David Sanchez Gomez  
Escuela Colombiana de Ingenieria Julio Garavito  
SPTI - Seguridad y Privacidad de Tecnologias de la Informacion  
Bogota, Colombia  
Email: {andersson.sanchez, cristian.pedraza-r, jeisson.sanchez}@escuelaing.edu.co

**Abstract**—LogiFlow is a real-time microservices platform for fleet routing where operational continuity and data integrity depend on trustworthy service interactions. This paper presents a complete DevSecOps security monitoring architecture that compares a vulnerable baseline against a hardened production-oriented implementation. We identify attack surfaces in API, WebSocket, webhook, container runtime, and CI/CD supply-chain paths, and prioritize threats through STRIDE and OWASP Top 10 (2021). The architecture integrates shift-left controls (CodeQL, Semgrep, npm audit, GitLeaks, Trivy, OWASP ZAP) with shift-right telemetry (Falco, Prometheus, Loki, Grafana) and distributed tracing (OpenTelemetry and Tempo). Evaluation using TC-01 to TC-10 demonstrates mitigation of high-risk scenarios such as unauthorized socket access, forged webhook injection, and request-flood abuse, while also improving triage speed through metric-log-trace correlation. The resulting artifacts form a reusable blueprint for academic and industry teams implementing continuous security in distributed systems.

**Index Terms**—DevSecOps, Microservices Security, OWASP Top 10, STRIDE, Runtime Monitoring, Falco, Prometheus, Loki, Grafana, OpenTelemetry, Tempo

## I. INTRODUCTION

Microservices increase agility and horizontal scalability, but they also expand trust boundaries and failure modes [1], [2]. In transportation and logistics platforms, a single compromised component can cascade into routing disruption, data exposure, or dispatch paralysis.

This work addresses the SPTI objective of security-by-design with measurable evidence, implementing and validating a full DevSecOps security layer over LogiFlow. The main contributions are:

- 1) A reproducible vulnerable-to-hardened architecture comparison with explicit vulnerability-to-control traceability.
- 2) An integrated CI/CD security pipeline with SAST, SCA, secrets detection, container scanning, and DAST.
- 3) A continuous monitoring and triage architecture that correlates metrics, logs, runtime anomalies, and traces.

## II. BACKGROUND

### A. Security and Governance Foundations

OWASP Top 10 (2021) is used as the vulnerability taxonomy [3]. STRIDE threat modeling provides a structured lens over spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege [4].

NIST CSF 2.0 and SSDF align governance with engineering execution, requiring traceable controls and repeatable verification [5], [6]. Additional control baselines and weakness references are mapped through NIST SP 800-53 and CWE knowledge bases [7], [8].

### B. Tooling and Observability Ecosystem

The implementation combines static and dynamic controls in CI (CodeQL, Semgrep, Trivy, ZAP) [9]–[12] and continuous runtime telemetry through Falco, Prometheus, Loki, Grafana, OpenTelemetry, and Tempo [13]–[18].

## III. LOGIFLOW SYSTEM DESCRIPTION

### A. Service Topology and Technology Stack

LogiFlow includes gateway, realtime, optimizer, automation, Redis, and PostgreSQL services. Authentication is JWT-based [19]; low-latency updates use Socket.IO [20]; data and event storage rely on PostgreSQL and Redis [21], [22].

### B. Trust Boundaries and Exposure Paths

The highest-risk boundaries are (i) public API ingress, (ii) websocket room joins, (iii) webhook ingestion, (iv) container runtime behavior, and (v) CI supply-chain dependencies.

## IV. THREAT MODEL

Threats are prioritized using:

$$R = L \times I \quad (1)$$

where  $L$  is likelihood (1–5) and  $I$  is impact (1–5).

Mitigation was staged by risk priority (P1 before P2/P3), preserving traceability to OWASP, STRIDE, and NIST.

TABLE I  
CORE LOGIFLOW SERVICES AND SECURITY-RELEVANT ROLE

| Service    | Stack                                         | Security-Relevant Role                                                          |
|------------|-----------------------------------------------|---------------------------------------------------------------------------------|
| Gateway    | NestJS, JWT                                   | API boundary, token issuance, webhook policy enforcement, security metrics      |
| Realtime   | Node.js, Socket.IO                            | Vehicle channel delivery, socket authentication, unauthorized-attempt telemetry |
| Optimizer  | Node.js, gRPC                                 | Route computation over internal service interfaces                              |
| Automation | n8n + webhook                                 | External event ingestion and orchestration trigger path                         |
| Monitoring | Falco, Prometheus, Loki, Grafana, Tempo, OTel | Detection, observability, and trace-based triage                                |

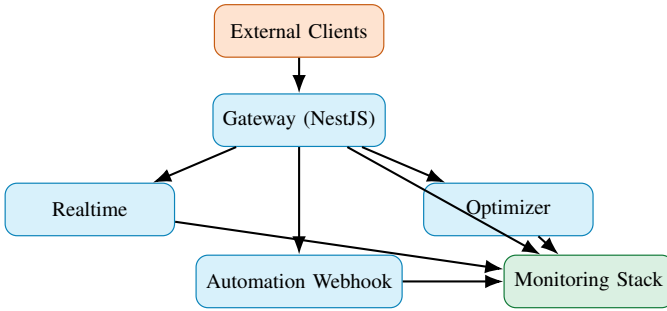


Fig. 1. LogiFlow architecture and security-observable boundaries.

TABLE II  
HIGHEST-PRIORITY THREATS (STRIDE EXCERPT)

| ID    | Threat Scenario                                         | L | I | Score |
|-------|---------------------------------------------------------|---|---|-------|
| TM-01 | Unauthorized subscription to vehicle socket rooms       | 5 | 5 | 25    |
| TM-02 | Forged webhook traffic events alter routing decisions   | 4 | 5 | 20    |
| TM-03 | Webhook request flood degrades service availability     | 5 | 4 | 20    |
| TM-08 | Malicious process execution inside container runtime    | 4 | 5 | 20    |
| TM-11 | Exploitation of vulnerable dependencies in exposed path | 4 | 4 | 16    |

## V. UNSECURE VS SECURE ARCHITECTURE COMPARISON

The secure architecture also removed dashboard “no data” blind spots by introducing baseline metric series and improving query defaults.

## VI. DEVSECOPS PIPELINE IMPLEMENTATION

### A. Pipeline Structure

The CI workflow runs on push, pull request, schedule, and manual dispatch. It includes distinct jobs for secrets, SCA, SAST, container scan, DAST, and summary reporting.

### B. Implemented Gates and Recent Reliability Fixes

- **SAST:** CodeQL and Semgrep for JavaScript/TypeScript.

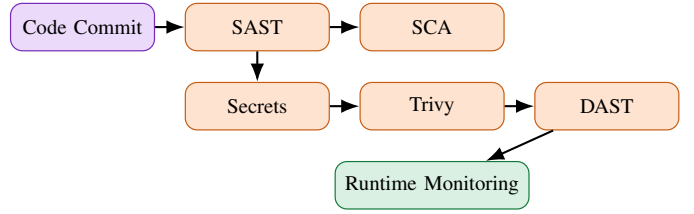


Fig. 2. Security pipeline from shift-left analysis to runtime observability.

- **SCA:** per-service dependency audit artifacts.
- **Secrets:** GitLeaks baseline in CI.
- **Container Security:** Trivy over gateway Docker context. A non-root user was added to satisfy DS-0002.
- **DAST:** ZAP baseline and parser-based gating against gateway target URL.
- **CI Hardening:** automatic creation of external Docker network before DAST compose startup.

## VII. CONTINUOUS MONITORING AND TRIAGE STACK

### A. Metrics, Logs, Runtime Alerts, and Traces

The production-like monitoring stack includes Falco, Prometheus, Loki, Grafana, Tempo, and OpenTelemetry Collector. Gateway and realtime services publish custom security metrics and distributed traces.

### B. Dashboards and Navigation Enhancements

The final implementation includes three security dashboards, one dedicated traces dashboard, and panel-level links from high-signal panels (Auth Failures and Falco Critical Alerts) to Grafana Explore with Tempo preselected. This reduced analyst click-depth during triage.

## VIII. EVALUATION AND RESULTS

### A. Validation Coverage

The security test plan (TC-01 to TC-10) covers authentication bypass, socket abuse, DoS, injection, header validation, secrets scanning, dependency checks, runtime anomaly detection, JWT tampering, and error-disclosure probing.

### B. Observed Security Posture Improvements

Compared to the vulnerable baseline, the hardened implementation provided:

- Stronger prevention against external exploit paths (API, socket, webhook).
- Earlier defect detection in CI through layered static and dynamic gates.
- Better incident forensics through correlated metrics, logs, runtime events, and traces.
- Improved pipeline reliability after resolving Trivy non-root policy and DAST network bootstrap issues.

TABLE III  
BASELINE VULNERABILITY TO IMPLEMENTED SECURITY CONTROL MAPPING

| ID    | Baseline Weakness                                | Implemented Control                                                      | Verification                            |
|-------|--------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------|
| V-001 | Unauthenticated Socket.IO room subscription      | JWT middleware enforced before join, with unauthorized-attempt metricing | Socket injection scenario rejected      |
| V-002 | Webhook endpoint accepted unauthenticated events | Bearer-protected webhook ingestion in gateway and automation flow        | Unauthorized requests return HTTP 401   |
| V-003 | No webhook rate limiting                         | Request throttling with alertable counters                               | Flood scenario produces HTTP 429        |
| V-005 | Missing gateway hardening headers                | Helmet and secure bootstrap configuration                                | Header validation by curl and DAST      |
| V-006 | Verbose internal error exposure                  | Production-safe exception handling                                       | Sanitized error body in production mode |
| V-007 | Vulnerable dependency exposure                   | CI SCA artifacts and policy evaluation                                   | Critical/high CVE trend monitoring      |
| V-008 | No runtime anomaly detection                     | Falco rules + Loki/Grafana runtime triage                                | Suspicious container behavior alerts    |

TABLE IV  
REPRESENTATIVE SECURITY TELEMETRY (GROUPED)

| Group           | Security Meaning                                                   |
|-----------------|--------------------------------------------------------------------|
| Auth Metrics    | Failed logins, JWT rejections, and unauthorized socket attempts    |
| Webhook Metrics | Event-ingestion activity and abuse-rate indicators                 |
| Runtime Alerts  | Falco syscall-level anomaly events and critical rule hits          |
| Trace Data      | Cross-service latency, root service identity, and span correlation |

TABLE V  
VALIDATION SUMMARY FOR CORE SECURITY SCENARIOS

| Test  | Security Objective                                | Result |
|-------|---------------------------------------------------|--------|
| TC-01 | Enforce authentication on protected endpoints     | Pass   |
| TC-02 | Reject unauthorized socket joins                  | Pass   |
| TC-03 | Trigger rate-limit under flood conditions         | Pass   |
| TC-04 | Detect SQL injection indicators via DAST baseline | Pass   |
| TC-05 | Validate secure header posture                    | Pass   |
| TC-06 | Detect leaked credentials in repository           | Pass   |
| TC-07 | Track dependency vulnerabilities by service       | Pass   |
| TC-08 | Detect suspicious container behavior at runtime   | Pass   |
| TC-09 | Reject manipulated JWT privilege escalation       | Pass   |
| TC-10 | Avoid internal error leakage in production mode   | Pass   |

### C. Limitations

The current implementation emphasizes practical reproducibility over enterprise-scale hardening. Remaining gaps include managed secret rotation, formal SOAR playbooks, and long-horizon benchmarking for MTTD/MTTR under sustained attack simulation.

## IX. SPTI COURSE CONCEPTS MAPPING

## X. CONCLUSIONS AND FUTURE WORK

This paper presented a complete, evidence-driven DevSecOps monitoring architecture for LogiFlow. The transition from vulnerable baseline to hardened implementation reduced ex-

ploitable paths and improved operational visibility for security response.

Future work includes: (i) vault-backed and automated secret rotation, (ii) SOAR-assisted response automation for Falco and Prometheus alerts, (iii) policy-as-code maturity for supply-chain governance, and (iv) quantitative longitudinal benchmarking of detection and response performance.

## ACKNOWLEDGMENT

This work was developed in the SPTI course at Escuela Colombiana de Ingenieria Julio Garavito.

## REFERENCES

- [1] S. Newman, *Building Microservices*. O'Reilly Media, 2015.
- [2] N. Dragoni, S. Giallorenzo, A. Lluch-Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216.
- [3] "Owasp top 10: The ten most critical web application security risks," <https://owasp.org/www-project-top-ten/>, 2021, accessed: 2026-04-09.
- [4] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
- [5] "Nist cybersecurity framework (csf) 2.0," <https://www.nist.gov/cyberframework>, 2024, accessed: 2026-04-09.
- [6] "Nist secure software development framework (ssdf) sp 800-218," <https://csrc.nist.gov/publications/detail/sp/800-218/final>, 2022, accessed: 2026-04-09.
- [7] "Nist sp 800-53 rev. 5: Security and privacy controls for information systems and organizations," <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>, 2020, accessed: 2026-04-09.
- [8] "Mitre common weakness enumeration (cwe)," <https://cwe.mitre.org/>, 2026, accessed: 2026-04-09.
- [9] "Codeql documentation," <https://codeql.github.com/docs/>, 2026, accessed: 2026-04-09.
- [10] "Semgrep documentation," <https://semgrep.dev/docs/>, 2026, accessed: 2026-04-09.
- [11] "Trivy documentation," <https://aquasecurity.github.io/trivy/>, 2026, accessed: 2026-04-09.
- [12] "Owasp zed attack proxy (zap) documentation," <https://www.zaproxy.org/docs/>, 2026, accessed: 2026-04-09.
- [13] "Falco runtime security documentation," <https://falco.org/docs/>, 2026, accessed: 2026-04-09.
- [14] "Prometheus documentation," <https://prometheus.io/docs/>, 2026, accessed: 2026-04-09.
- [15] "Grafana loki documentation," <https://grafana.com/docs/loki/latest/>, 2026, accessed: 2026-04-09.
- [16] "Grafana documentation," <https://grafana.com/docs/grafana/latest/>, 2026, accessed: 2026-04-09.

TABLE VI  
SPTI CONCEPT-TO-ARTIFACT TRACEABILITY

| SPTI Concept                     | Evidence in This Work                                                                            | Practical Outcome                                        |
|----------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Risk Management                  | STRIDE register with risk equation and priority bands                                            | Structured mitigation order for high-impact threats      |
| Security Architecture Principles | JWT guards, bearer policy, rate limiting, exception hardening, least-privilege container runtime | Lower attack success probability across trust boundaries |
| SGSI / Governance                | NIST-mapped controls matrix, CI evidence artifacts, repeatable test criteria                     | Auditability and traceable control ownership             |
| Data Protection                  | Secrets detection and reduced unauthorized telemetry exposure                                    | Improved confidentiality and accountability              |
| Software Security                | CodeQL, Semgrep, Trivy, ZAP, dependency audit, runtime alerts                                    | Continuous quality gates and faster remediation          |
| Continuous Monitoring            | Falco + Prometheus + Loki + Grafana + Tempo + OpenTelemetry                                      | End-to-end detection and correlated triage               |

- [17] “Opentelemetry documentation,” <https://opentelemetry.io/docs/>, 2026, accessed: 2026-04-09.
- [18] “Grafana tempo documentation,” <https://grafana.com/docs/tempo/latest/>, 2026, accessed: 2026-04-09.
- [19] M. Jones *et al.*, “Rfc 7519: Json web token (jwt),” <https://www.rfc-editor.org/rfc/rfc7519>, 2015, accessed: 2026-04-09.
- [20] “Socket.io documentation,” <https://socket.io/docs/v4/>, 2026, accessed: 2026-04-09.
- [21] “Postgresql documentation,” <https://www.postgresql.org/docs/>, 2026, accessed: 2026-04-09.
- [22] “Redis documentation,” <https://redis.io/docs/>, 2026, accessed: 2026-04-09.